Decomposition in Normalization

Decomposition is the process of breaking a complex relation (table) into two or simpler relations to eliminate redundancy, resolve anomalies, and ensure data integrity—often as part of normalization.

It helps organize the data into smaller, more manageable tables while maintaining the ability to reconstruct the original table using joins.

There are two main types:

1. Lossless (or Lossless-Join) Decomposition
    - Essential goal: No data is lost after decomposition.
    - You can reconstruct the original table by joining the smaller tables.
    - This is the preferred and safe form of decomposition.
2. Lossy Decomposition
    - Some information is lost when decomposing and recombining.
    - Occurs when tables are not properly related using keys.
    - Generally undesirable and should be avoided.

Decomposition is used to:

- Remove data redundancy
- Eliminate insertion, update, and deletion anomalies
- Make the structure easier to understand and maintain
- Satisfy the rules of normal forms

Example of Decomposition

Suppose you have a table:

StudentID	StudentName	CourseName	Instructor
1	Alice	DBMS	Dr. Smith
2	Bob	DBMS	Dr. Smith
1	Alice	OS	Dr. Jones

Problems:

- Repetition of student and instructor names
- Redundant data
- Update anomalies (changing "Dr. Smith" requires multiple updates)

Decomposition (into 3NF):

1. **Student Table**

StudentID	StudentName
1	Alice
2	Bob

2. **Course Table**

CourseName	Instructor
DBMS	Dr. Smith
OS	Dr. Jones

3. **Enrollment Table**

StudentID	CourseName
1	DBMS
2	DBMS
1	OS

These can be joined to reproduce the original table **without any data loss**.

Key Benefits of Decomposition

- Avoids anomalies
- Reduces storage space
- Improves data consistency
- Makes the design modular and easier to maintain

De-normalization

De-normalization is the intentional process of introducing redundancy into a database by combining tables that were separated during normalization. It's the reverse of normalization, used mainly to improve performance in read-heavy systems—especially when querying joins frequently slows down the system.

While normalization improves consistency and reduces redundancy, too much normalization can hurt performance. De-normalization helps when:

- Queries involve complex joins across many tables.
- The system is read-heavy (many SELECTs, fewer INSERTs/UPDATEs).
- Reporting and analytics require frequent aggregation of data.
- You want to reduce the number of joins in queries to make them faster.

Example of De-normalization

Let's revisit our normalized schema from earlier:

Normalized Tables:

Students

StudentID	StudentName
1	Alice

Courses

CourseName	Instructor
DBMS	Dr. Smith

Enrollments

StudentID	CourseName
1	DBMS

Denormalized Table:

StudentID	StudentName	CourseName	Instructor
1	Alice	DBMS	Dr. Smith

Faster query (you don't need to JOIN 3 tables)
Data redundancy (if "Dr. Smith" changes, you must update it in multiple rows)

Normalization vs. De-normalization

Feature	Normalization	De-normalization
Redundancy	Reduced	Increased
Query Performance	May require joins (slower)	Fewer joins (faster for reads)
Update Performance	Better consistency	Risk of anomalies
Storage Use	Less space	More space due to repetition
Complexity	More tables and relationships	Fewer tables, but riskier to manage

When to Use De-normalization

- For data warehouses or OLAP systems (analytics-heavy)
- When query speed is critical and updates are rare
- When hardware resources can handle the extra storage